sabrith-tech / ewb-secure-vpc-peering

Code Issues Pull requests Agents Actions Projects Wiki Security and quality

Insights Settings

main ewb-secure-vpc-peering / docs / Sabrith_EWB_VPC_Peering.md

sabrith-tech Update test phases in architecture validation matrix aa41006 · 3 minutes ago

64 lines (46 loc) · 4.19 KB

Preview Code Blame Raw

# AWS Secure VPC Peering: Architectural Report & Lab Workbook

- **Engineer:** Sabrith
- **Location:** Asia Pacific (Mumbai) `ap-south-1`
- **Date:** 2026-10-30 UTC
- **Project Status:** COMPLETED & VERIFIED

## 1. Executive Summary

This workbook documents the design, implementation, and multi-stage testing of a secure, multi-VPC network infrastructure in AWS. The architecture segregates network layers into public gateways and isolated private compute subnets across different VPCs.

To bridge these environments securely, VPC Peering was implemented alongside custom subnet route tables and granular security group rules. Live traffic testing verified that connectivity is governed by least-privilege security policies, keeping critical resources hidden from public exposure while allowing authorized administrative channels.

## 2. Technical Inventory & Specifications

### A. Network Architecture

- **VPC Alpha CIDR:** `10.10.0.0/16` (ID: `vpc-00759f9a840911a3b` )
- **VPC Beta CIDR:** `10.20.0.0/16` (ID: `vpc-YOUR_BETA_VPC_ID` )
- **VPC Peering Connection:** `ewb-alpha-beta-pcx` (Status: Active)

### B. Subnet Allocations

1. `ewb-alpha-public-subnet-a` | `10.10.1.0/24` | `ap-south-1a` | Public Gateway
2. `ewb-alpha-private-subnet-a` | `10.10.2.0/24` | `ap-south-1a` | Alpha App Layer
3. `ewb-beta-private-subnet-a` | `10.20.1.0/24` | `ap-south-1a` | Beta App Layer

### C. Compute Instances

- **ewb-bastion:** `10.10.1.10` (Public IP: `13.232.163.2` Redacted) | `t3.micro` | `ewb-bastion-sg`
- **ewb-app-a:** `10.10.2.10` (Private) | `t3.micro` | `ewb-app-a-sg`
- **ewb-app-b:** `10.20.1.10` (Private) | `t3.micro` | `ewb-app-b-sg`

## 3. Architecture Validation Matrix

The network's security boundaries were validated using six explicit test phases:

| Test Phase | Source Node | Destination Node | Port / Protocol | Expected State | Observed State |
|---|---|---|---|---|---|
| 1. Baseline Isolation | App-A ( `10.10.2.10` ) | App-B ( `10.20.1.10` ) | ICMP (Ping) | 100% Packet Loss | 100% Packet Loss (Silence) |
| 2. Forward Peering | App-A ( `10.10.2.10` ) | App-B ( `10.20.1.10` ) | ICMP (Ping) | 0% Packet Loss | 0% Packet Loss (Replies Active) |
| 3. Reverse Peering | App-B ( `10.20.1.10` ) | App-A ( `10.10.2.10` ) | ICMP (Ping) | 0% Packet Loss | 0% Packet Loss (Replies Active) |
| 4. Security Block | App-A ( `10.10.2.10` ) | App-B ( `10.20.1.10` ) | ICMP (Rule Deleted) | 100% Packet Loss | 100% Packet Loss (Silence) |
| 5. Path Recovery | App-A ( `10.10.2.10` ) | App-B ( `10.20.1.10` ) | ICMP (Rule | 0% Packet Loss | 0% Packet Loss |

| Test Phase | Source Node | Destination Node | Port / Protocol | Expected State | Observed State |
|---|---|---|---|---|---|
| | | | Restored) | | (Replies Restored) |
| 6. Secret Mission | Bastion ( `10.10.1.10` ) | App-B ( `10.20.1.10` ) | SSH (Cross-VPC SG Ref) | Active Shell Connection | Active Shell Connection (ProxyJump verified) |

## 4. Key Engineering Takeaways

### A. The Separation of Networking & Firewalls

This lab successfully isolated the difference between network *routing* (VPC Peering and Route Tables) and instance-level *security policy* (Security Groups). While VPC Peering acts as the physical bridge connecting the subnets, security groups enforce the logical firewall rules. Even with an active peer connection, traffic is securely blocked unless the target security group explicitly permits it.

### B. SSH Key Security on Windows

To establish secure ProxyJump administration, strict OS-level permissions were enforced on the SSH private key. By disabling Windows inheritance and removing the `SYSTEM` and `Administrators` security groups, the `.pem` file was restricted to a single authorized user (Admin). This prevents unauthorized key reading and satisfies OpenSSH requirements.

## 5. Safe Decommissioning Plan

To maintain strict cost management and avoid unnecessary charges on the AWS account:

1. **Terminate EC2 Instances:** Terminate `ewb-bastion` , `ewb-app-a` , and `ewb-app-b` to release compute resources and terminate billing for public IPv4 allocation.
2. **Delete VPC Peering:** Delete the `ewb-alpha-beta-pcx` connection to tear down the bridge.
3. **Clean Up Route Tables & Subnets:** Delete custom subnets and route tables.
4. **Delete VPCs:** Delete `ewb-alpha-vpc` and `ewb-beta-vpc` to return the regional workspace to its pristine, zero-cost state.